AI ENGINEERING ASSESSMENT · INFERENCE GROUP

EventOps Intelligence.

Full-Stack AI Operations, Retrieval-Augmented Generation & Agentic Verification

EXECUTIVE TECHNICAL BRIEF

AUTHOR	Keeno Smith
CLASSIFICATION	Full-stack AI application / Agentic automation platform
DOMAIN	Event operations, data verification and knowledge intelligence
ARCHITECTURE	React + FastAPI + PostgreSQL / pgvector + LangGraph + RAG
KEY AI COMPONENTS	FastEmbed, semantic retrieval, Gemini grounded generation
INTEGRATIONS	HTTPS Calendar / iCalendar, Mailtrap Email Testing, Web APIs
INFRASTRUCTURE	Render static site + Python service; Neon PostgreSQL
VERIFIED RELEASE	**7cc6568** · production assessment acceptance
DATE	11 October 2026

A deployed, evidence-grounded application connecting structured knowledge retrieval with auditable Calendar, Email and Web verification.

Live application https://eventops-intelligence.onrender.com

Source repository https://github.com/keenosmith-del/eventops-intelligence-v.01

Companion document: EventOps Local Reviewer Runbook. A separate portfolio showcase may be included with the submission.

01 Executive overview

EventOps Intelligence is a deployed full-stack application demonstrating two connected AI engineering capabilities: evidence-grounded answers over structured event data, and automated operational reconciliation across external systems. The system was developed for the Inference Group AI Engineer assessment and is accessible through a browser with scoped demonstration access.

Part 1 - Knowledge Intelligence. A conversational RAG assistant answers questions about events, operational policies and participant records. Instead of relying on general model knowledge, it retrieves relevant evidence from a controlled, structured knowledge base, retains conversational context and exposes the source records behind each result.

Part 2 - Operational Intelligence. A LangGraph workflow reads Calendar, Email and Web observations, extracts and normalises data, identifies duplicates, checks identities against authoritative records and persists verified or flagged outcomes. The reviewer can inspect execution evidence, reopen history and export CSV data.

> **Design principle.** Relevant information is not automatically verified information. Semantic retrieval selects evidence; generative AI can explain it; deterministic rules decide whether operational observations agree with trusted reference records.

Business problem

Event operations commonly span spreadsheets, calendars, registration emails and external websites. Different systems can disagree about names, event times, registration status or locations. Manual reconciliation is repetitive; language models can introduce a separate failure mode by presenting an unsupported answer confidently.

The application is designed to reduce that ambiguity: keep evidence linked to its source, distinguish accepted records from missing or contradictory information, and make the path from raw observation to final output inspectable. These are architectural aims, not claims of measured production efficiency.

Why a full-stack implementation

I chose React, FastAPI, PostgreSQL and LangGraph over a purely low-code implementation because my engineering background spans frontend development, backend services, databases and AI integrations. A full-stack route gave me direct control over retrieval behaviour, model inputs, API contracts, persistence, tool execution, verification rules, error handling and reviewer experience.

Low-code orchestration remains useful for integrations and scheduling. For this assessment, LangGraph provides the principal stateful workflow, while a separate n8n integration remains optional and is not required for the core demonstration.

02 Architecture and engineering boundaries

The application separates browser interaction, API validation, AI processing, external tools and durable data storage. The React frontend does not connect directly to the language-model provider, Mailtrap credentials or PostgreSQL.

```
[React / Vite reviewer interface]
             |
[FastAPI: sessions, RAG, agents, exports]
        |                          |
  {RAG processing}          {LangGraph workflow}
        |                          |
 {FastEmbed + Gemini}    [Calendar | Email | Web]
        |                          |
 [PostgreSQL + pgvector: records, vectors, runs]
```

Technology responsibilities

React 19 / Vite / React Router / React Flow Render the reviewer interface, conversations, execution states, verification evidence and run history.

Python 3.12 / FastAPI / Pydantic Expose REST APIs, validate requests, coordinate backend services and keep external credentials server-side.

PostgreSQL / pgvector Persist knowledge records, 384-dimensional embeddings, conversation history, agent steps and verification outputs.

FastEmbed / BAAI/bge-small-en-v1.5 Generate embeddings locally without requiring a paid embedding API.

Gemini Generate natural-language answers from bounded retrieved evidence when explicitly enabled; validate cited source identifiers.

LangGraph Run an explicit state graph for external reads, extraction, normalisation, matching and classification.

Production and development environments

Production: Render hosts the static React site and FastAPI web service; Neon hosts the persistent PostgreSQL database with pgvector. The verified hosted engine was PostgreSQL 18.6. GitHub CI and Render deployments follow changes pushed to the project repository.

Local: React runs on 127.0.0.1:5173, FastAPI on 127.0.0.1:5050 and Docker PostgreSQL on 127.0.0.1:55432. Environment configuration and database records remain independent from production.

03 Part 1 - Retrieval-Augmented Generation

Structured knowledge and ingestion

The controlled assessment corpus contains **three structured documents and 19 records**: **events.csv** (3 event records), **policies.csv** (9 operational-policy records) and **directory.json** (7 participant records). Stable record identifiers and source metadata allow retrieved facts to be traced back to their origin.

The ingestion process parses and validates each format, normalises the information, applies source-aware record chunking and stores searchable semantic representations. FastEmbed produces 384-dimensional vectors using BAAI/bge-small-en-v1.5, and pgvector provides persistent similarity retrieval.

```
[CSV events / CSV policies / JSON directory]
                    |
     {Parse + validate + source-aware chunks}
                    |
       {FastEmbed: 384-dimensional vectors}
                    |
        [PostgreSQL / pgvector retrieval]
                    |
       {Query transformation + conversation}
                    |
     {Grounded Gemini + citation validation}
                    |
         [Answer + inspectable evidence]
```

Reasoning, context and source control

Before answering a question, the backend interprets supported event identifiers and retrieves the most relevant records. Stored conversation history helps resolve contextual follow-ups, such as asking about “its check-in” after discussing EVT-101. The original question and its retrieved evidence remain associated with the conversation.

When generative mode is enabled, Gemini receives bounded evidence and structured grounding instructions. Returned citation identifiers are checked against the specific records supplied to the model. This improves traceability but does not guarantee that every generated claim is factually entailed; broader claim-level evaluation is a future improvement.

Assessment requirement coverage - Part 1

Three structured documents Met: CSV events, CSV policies and JSON participant directory, totaling 19 indexed records.

Semantic retrieval Met: locally generated embeddings, PostgreSQL / pgvector search and ranked evidence.

Prompt control and context Met: event-aware query transformation, constrained model input, persistent conversations and contextual follow-ups.

Source traceability Met: source identifiers, retrieved evidence and validated citation references.

Web deployment Met: React and FastAPI are publicly deployed and connected to Neon.

Quality evidence The controlled retrieval evaluation passed 13/13 reference cases; production retrieval identified policy POL-003 and the correct EVT-101 check-in deadline of **08:45 SAST**.

Generative verification: One real Gemini response was produced in the hosted environment and returned the correct 08:45 deadline with a valid POL-003 citation. New generation was disabled again after that bounded test; ordinary reviewer access continues to support retrieval and evidence inspection.

04 Part 2 - Agent orchestration and verification

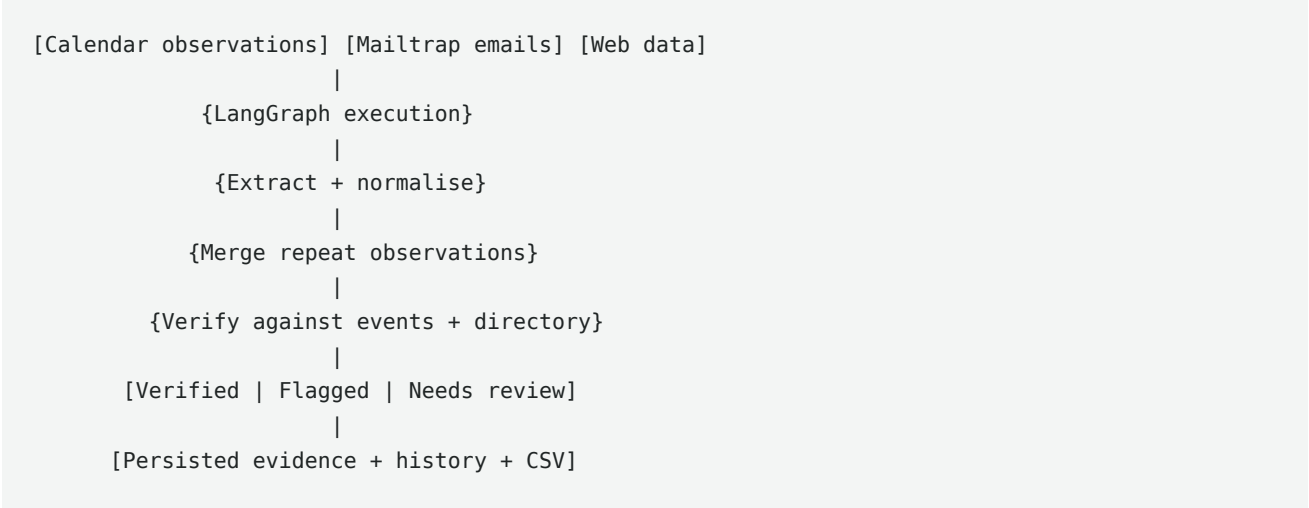
External tools and workflow

LangGraph coordinates a bounded operational workflow. It executes predefined tool-reading and processing stages after a reviewer starts a run. This is autonomous stage execution, not unrestricted LLM-driven tool selection. The constraint allows the system to retain predictable verification rules, structured run states and inspectable errors.

Calendar Retrieve real iCalendar information over HTTPS and parse recognised event and scheduling fields.

Email Read synthetic event-registration messages from the Mailtrap Email Testing sandbox API; the production assessment exercised non-empty message extraction.

Web Fetch real HTTPS reference observations. Public data may legitimately fail to match the fictional EVT dataset.



Extraction, cleansing and decision rules

Source-specific parsers transform recognised iCalendar, JSON and text content into comparable event or participant observations. Normalisation standardises supported names, email addresses, dates and text values. Duplicate evidence can be merged while retaining source references, whereas contradictory values remain available for review.

Reference verification checks event identity, participant status and recognised registration fields against authoritative synthetic records. A participant marked inactive cannot be accepted because an email repeats their name; an unrelated web record cannot be treated as a verified EVT attendee.

Assessment requirement coverage - Part 2

At least three external tools Met in production: real Calendar, Mailtrap Email Testing and Web HTTPS integrations.

Semi-structured extraction Met for supported calendar, email and reference-data formats; unrestricted attachment/OCR processing is outside v.01 scope.

Data-quality automation Met: normalisation, deduplication, matching and explicit missing/conflicting-record flags.

Verifiable output Met: persistent status, source evidence, execution stages, history and actual CSV downloads.

Explainable orchestration Met: stored LangGraph stages, statuses and outputs; deterministic rules remain responsible for verification outcomes.

05 Production acceptance evidence

The deployed release **7cc6568** passed the core hosted assessment workflows. Production verification tested a real Calendar/Email/Web execution and persisted its results in Neon PostgreSQL. Local and hosted checks were kept distinct.

Real three-source execution

A single production run processed **13 observations** (five Calendar, three Mailtrap messages and five Web observations) into **seven stored result records**. The email cases were purpose-built synthetic registrations against the controlled EVT-101 dataset:

Ada - positive registration Verified against active participant **P-001** and event **EVT-101**.

Ada - duplicate registration Merged with the positive record, retaining references to both original email messages.

Lindi - conflicting registration Flagged because participant **P-003** is inactive and the supplied name does not match the reference identity.

Unrelated public-calendar and web observations remained separately flagged rather than becoming invented positive registrations. The complete CSV contained seven result rows; the verified-only CSV contained one result row.

> **Production evidence:** run ID `ef50214e-9ec1-4903-a80c-ada28ca7d5cb`. Results, history and session-scoped persistence survived deployment. The workflow completed without resending the three sandbox messages.

Automated and hosted validation

- **217** backend tests passed.
- **16** frontend unit tests passed.
- **25** browser tests passed; the production build passed.
- **Four** opt-in live tests were skipped in automation; separate controlled production checks covered the critical integrations.
- RAG corpus: four migrations applied, pgvector available, all 19 source records embedded at 384 dimensions.
- Reviewer isolation, CSV export, restored history and contextual RAG follow-ups were verified against hosted data.

The reported results are bounded assessment checks, not a claim of general-world extraction accuracy, enterprise load capacity or guaranteed uptime.

06 Engineering quality, controls and limitations

Reliability and security decisions

Credential handling Provider tokens and database connection strings remain server-side; local and production secrets are separate.

Reviewer sessions Scoped demonstration access isolates conversations and agent runs across separate reviewer contexts; this is not production customer authentication.

Operational persistence PostgreSQL stores conversation and execution data beyond individual backend processes, while migrations and source indexing are repeatable.

Deterministic verification An LLM may compose an explanation, but reference records and explicit comparison rules decide verified versus flagged outcomes.

Tool safety External sources are read-only in the assessment workflow; outbound activity and request targets are constrained.

Current limitations

- Gemini generation was validated once in production but remains disabled for new requests by default.
- The initial RAG corpus is controlled. User-managed uploads and indexing of unfamiliar documents require additional feature verification.
- Extraction supports known structured and semi-structured patterns; arbitrary attachments and unrestricted model-assisted parsing are not implemented.
- The agent follows a predefined LangGraph execution path. Optional n8n integration is not configured as a hosted service.
- Long idle wake-up behaviour, precise Render memory headroom and the root cause of some earlier transient entry errors remain unverified.

Next version

A production-oriented iteration would add secure user-managed knowledge ingestion, schema-constrained extraction for varied messages and attachments, configurable verification policies, human approval for corrections, authenticated enterprise Calendar/Email connectors, more extensive retrieval evaluations, durable background workers, observability and stronger multi-user security. n8n could provide supplementary scheduling while LangGraph continues to own record verification.

07 Reviewer walkthrough and executive assessment

Recommended demonstration sequence

1 - Knowledge Assistant Ask “What time does check-in close for EVT-101?”, inspect POL-003 and try a contextual follow-up.

2 - Agent Workspace Run EVT-101 once or inspect a run stored in the current reviewer session. Review connected tool outcomes.

3 - Verification Results Inspect Ada’s verified identity and merged duplicate sources; inspect Lindi’s flagged inactive/name-mismatch evidence.

4 - CSV exports Download the all-record and verified-only CSV reports and compare them with the displayed results.

5 - Execution History Reopen a completed execution, inspect the saved stages and refresh to confirm persistence.

Executive assessment

EventOps Intelligence demonstrates full-stack AI engineering beyond a simple LLM call. The application combines structured knowledge processing, local embeddings, persistent semantic search, bounded generation, tool-based orchestration, deterministic verification, evidence storage, an interactive frontend and repeatable tests.

The central distinction remains deliberate: **AI-generated information and verified operational information are not interchangeable.** Users should be able to see which evidence was retrieved, what the agent processed and why a record was accepted or flagged. That separation is the basis for further adaptation to other business domains.

Application https://eventops-intelligence.onrender.com

Repository https://github.com/keenosmith-del/eventops-intelligence-v.01

Supporting document: EventOps Local Reviewer Runbook. Screenshots and any published walkthrough recording should be linked from the repository when available.